

Asignatura	Datos del alumno	Fecha
Analítica Escalable	Apellidos: ORTIZ GARCIA	10/05/2025
	Nombre: ASIER	

Actividad 8 – Apache Kafka con Confluent Cloud

1. Introducción

Esta actividad tiene como objetivo profundizar en el funcionamiento de Apache Kafka utilizando la plataforma Confluent Cloud. Durante la práctica se ha realizado una configuración completa que incluye la creación de un entorno personalizado, un clúster Kafka alojado en AWS, topics específicos para la generación de datos, y conectores automáticos Datagen para producir mensajes continuamente. También se ha integrado la herramienta Flink SQL Studio para realizar consultas avanzadas en tiempo real, demostrando así la capacidad analítica del ecosistema Kafka en la nube.

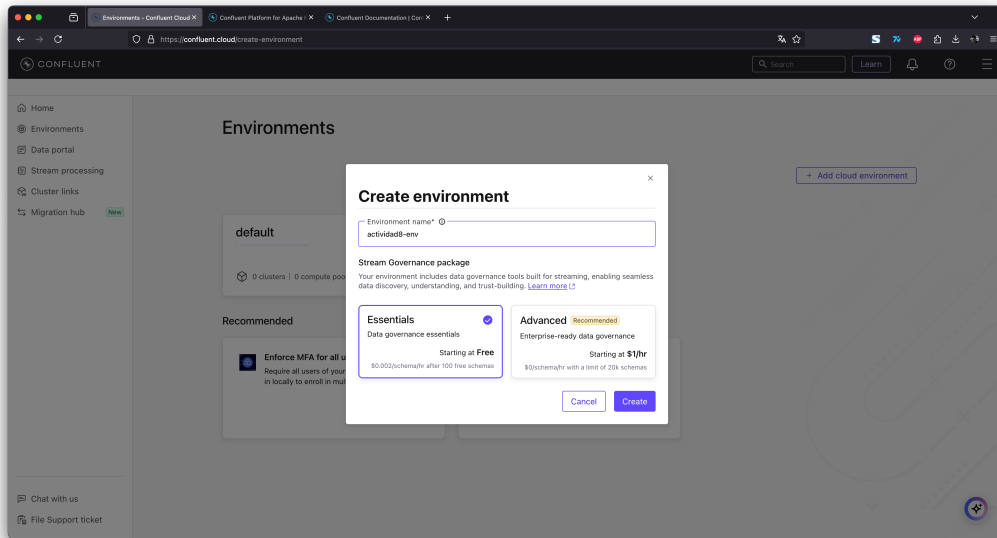
En una segunda parte, se ha complementado la configuración anterior mediante scripts en Python utilizando la librería confluent-kafka, permitiendo así la producción y consumo directo de eventos desde código. Esta implementación muestra de forma práctica la interacción vía API con Kafka, destacando aspectos clave como la distribución de particiones y el manejo de consumidores en grupo.

Toda la práctica se ha documentado paso a paso mediante capturas de pantalla detalladas, explicaciones técnicas claras, y enlaces al repositorio del código fuente para su consulta y ejecución posterior.

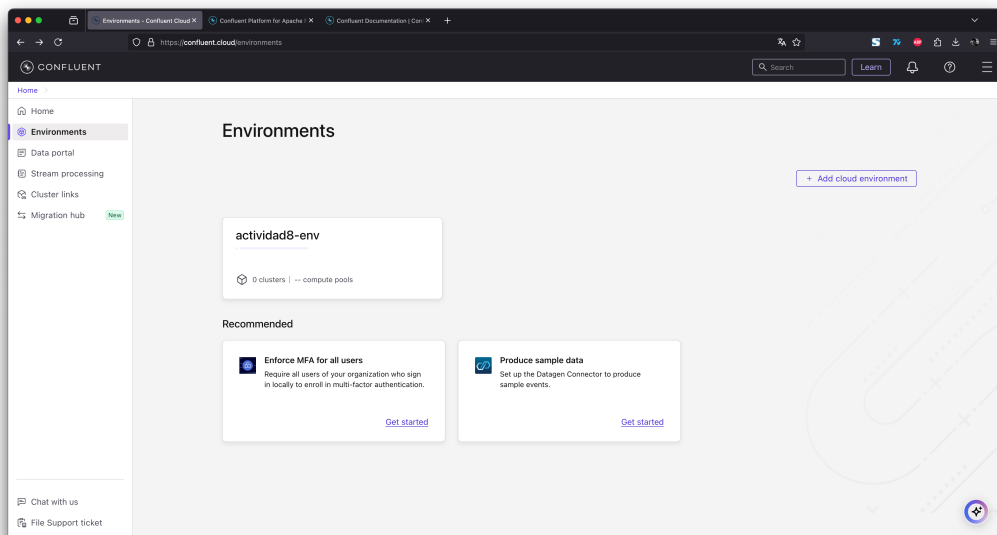
2. Parte 1 - Guía Confluent Cloud

Creación del entorno

Desde el sitio web de Confluent Cloud se crea un nuevo entorno de trabajo personalizado para la guía.



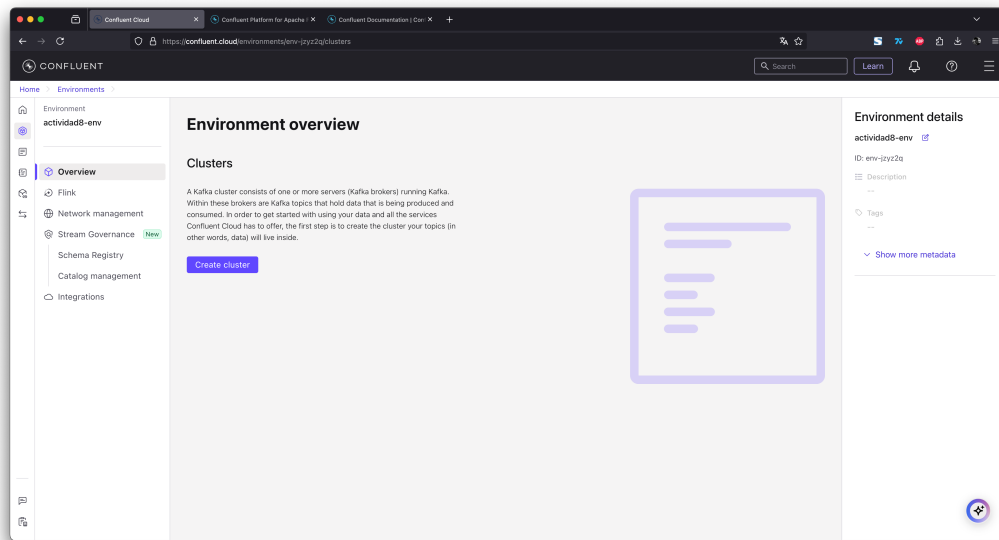
Creación de un nuevo entorno personalizado



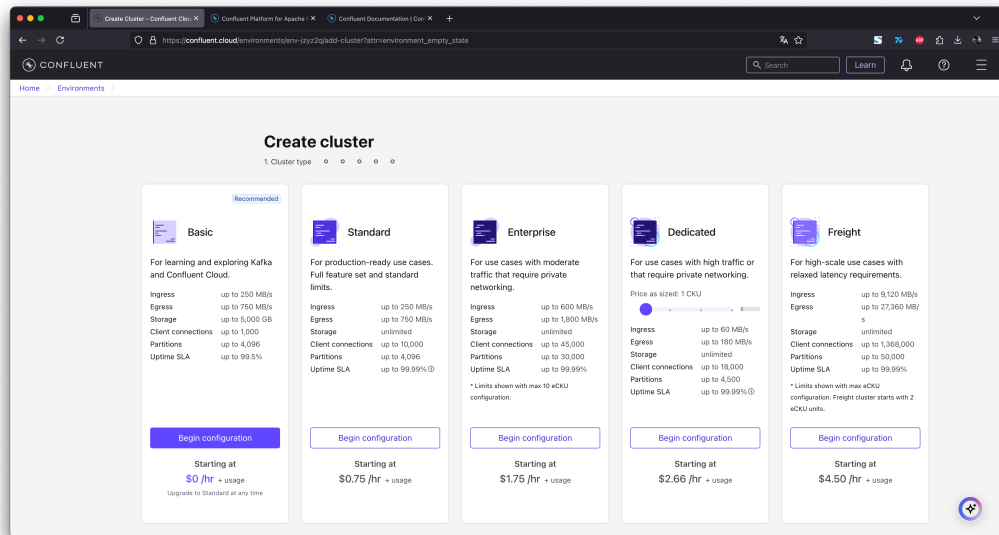
Confirmación del entorno ya creado correctamente

Creación del clúster

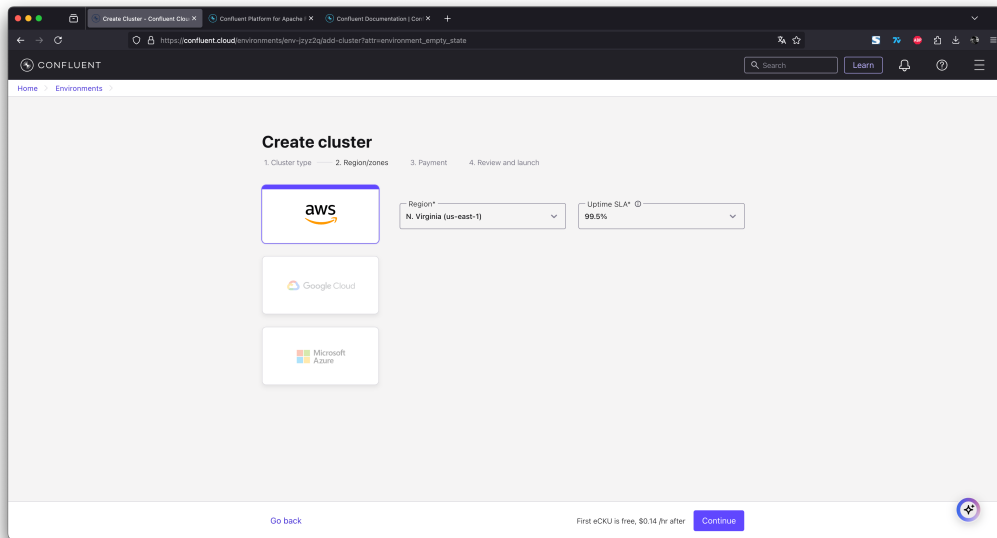
Una vez creado el entorno, se configura un clúster básico de Kafka en AWS.



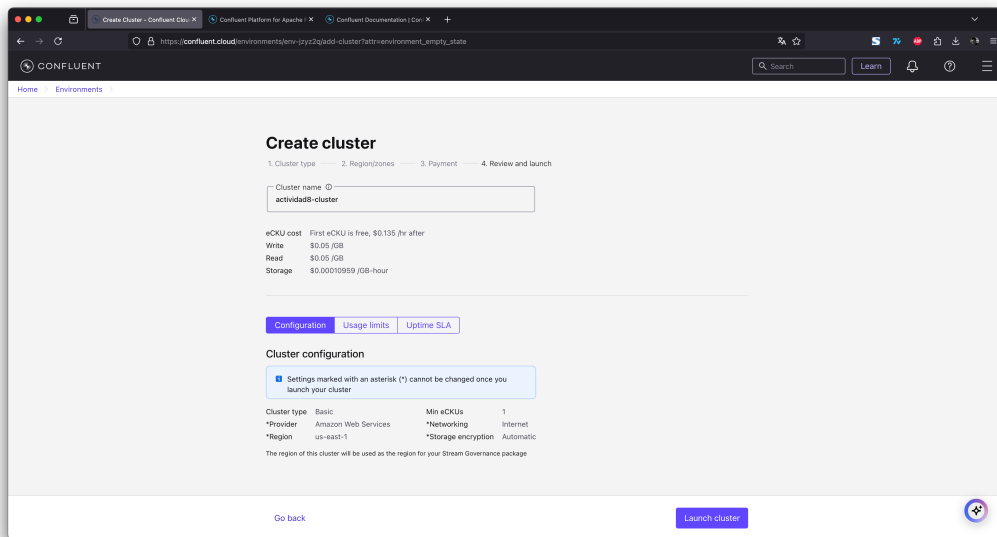
Página de inicio de creación del clúster



Selección de clúster básico



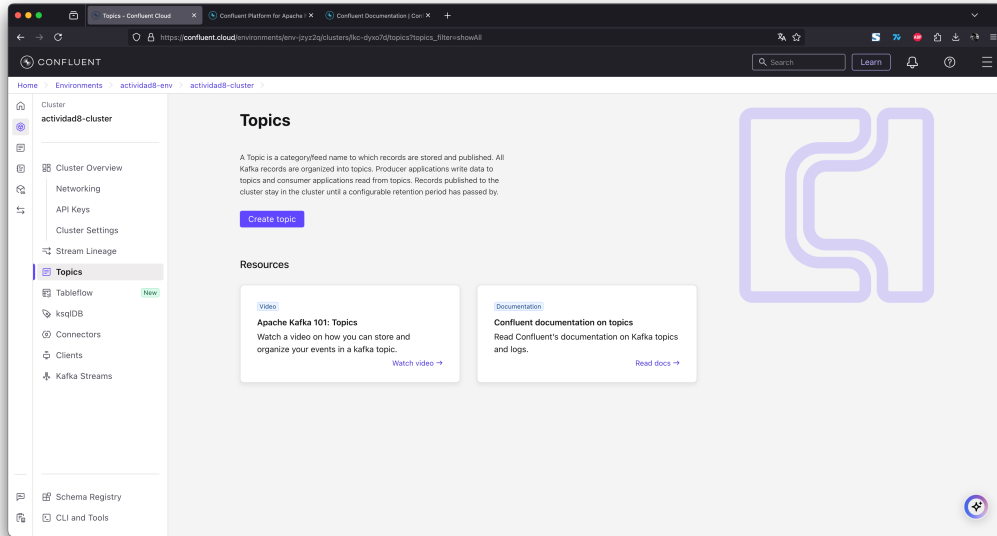
Selección del proveedor AWS y región us-east-1



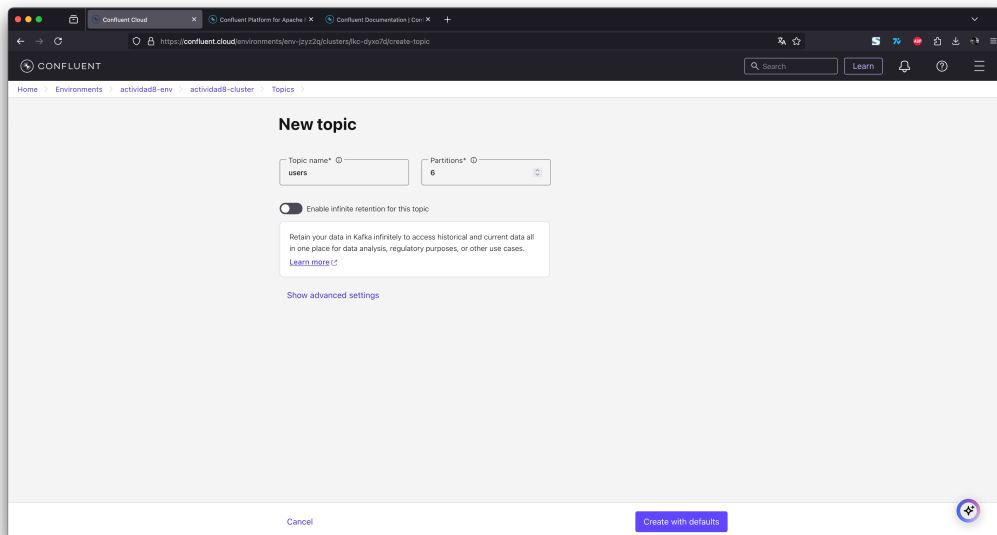
Resumen de configuración final antes de lanzar el clúster

Creación del topic

Se crea un topic llamado 'users' utilizando la configuración predeterminada proporcionada por la plataforma.



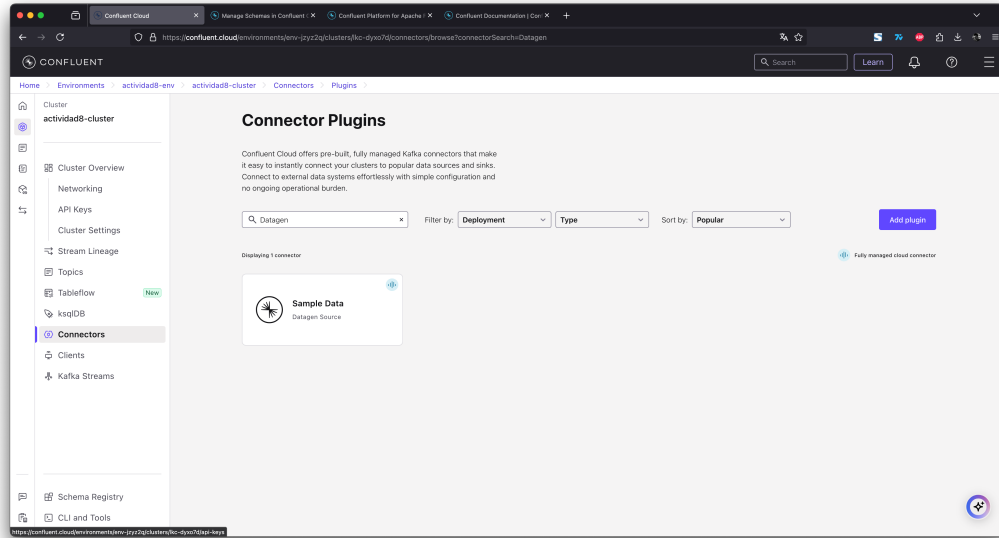
Pantalla de creación de un nuevo topic



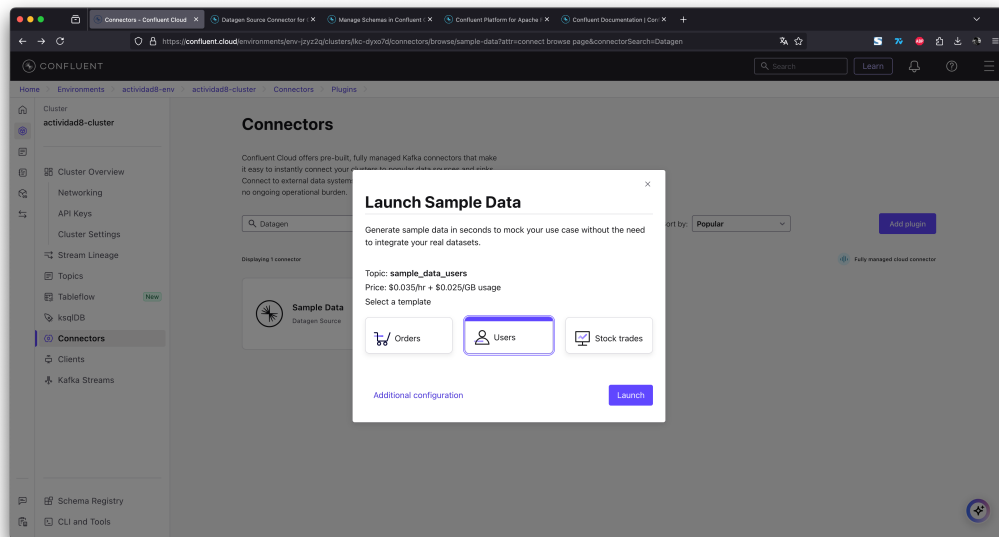
Creación del topic 'users' con la configuración predeterminada

Configuración del conector Datagen

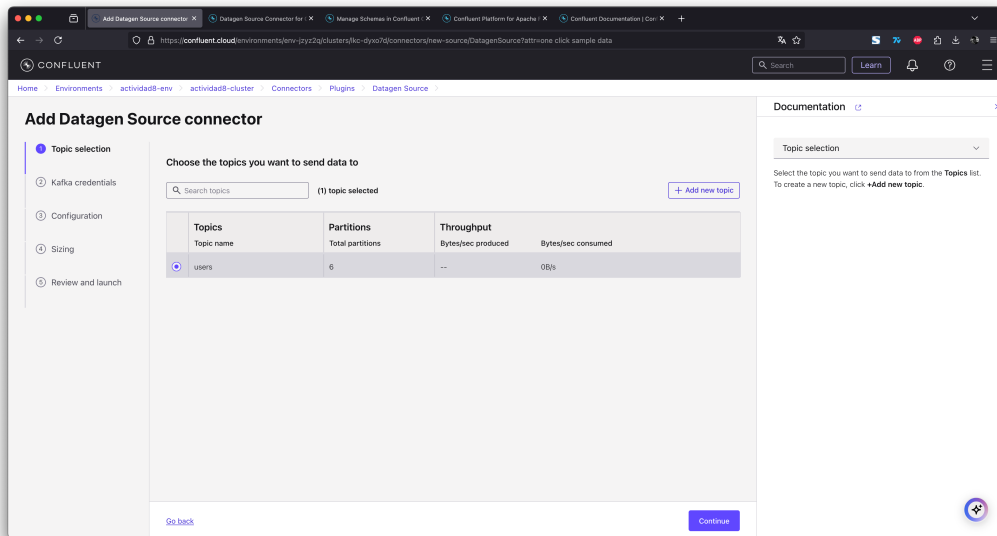
Se añade el conector Datagen para generar datos ficticios y enviarlos automáticamente al topic 'users'.



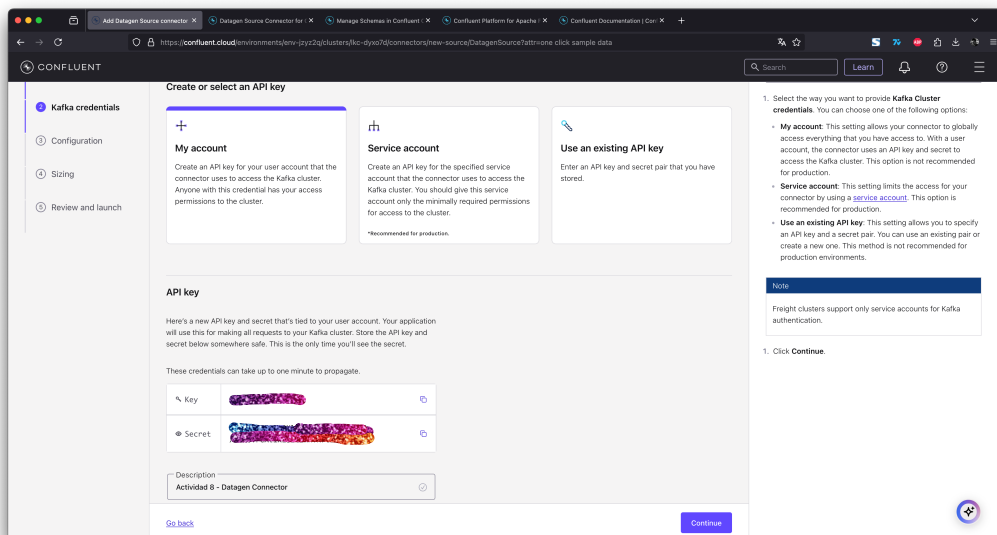
Acceso a la selección de integración de conectores



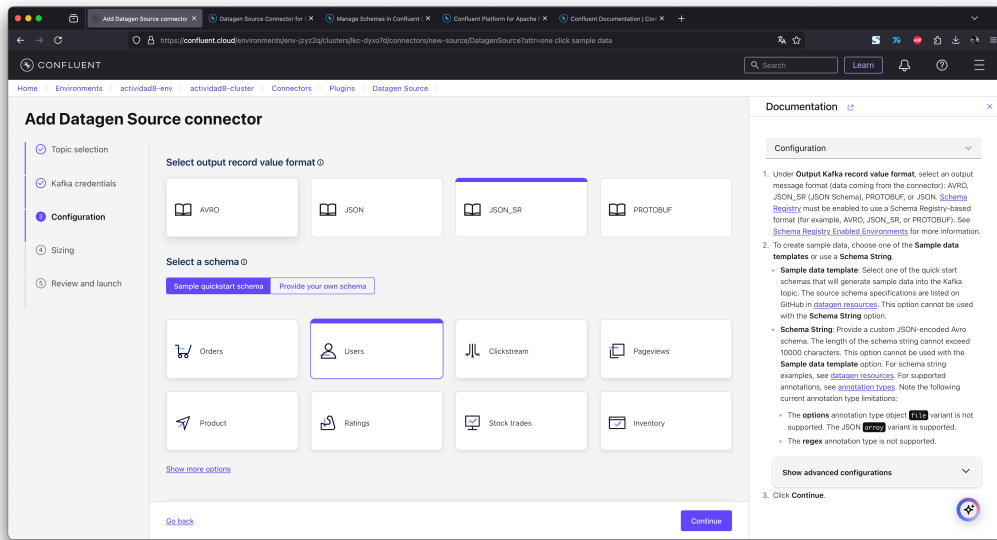
Selección del tipo de datos simulados



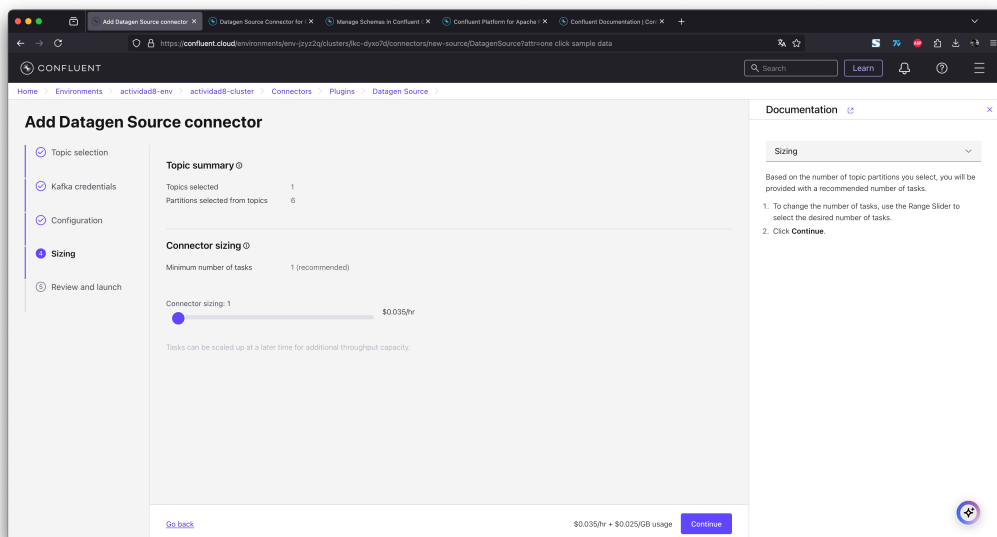
Confirmación del topic objetivo



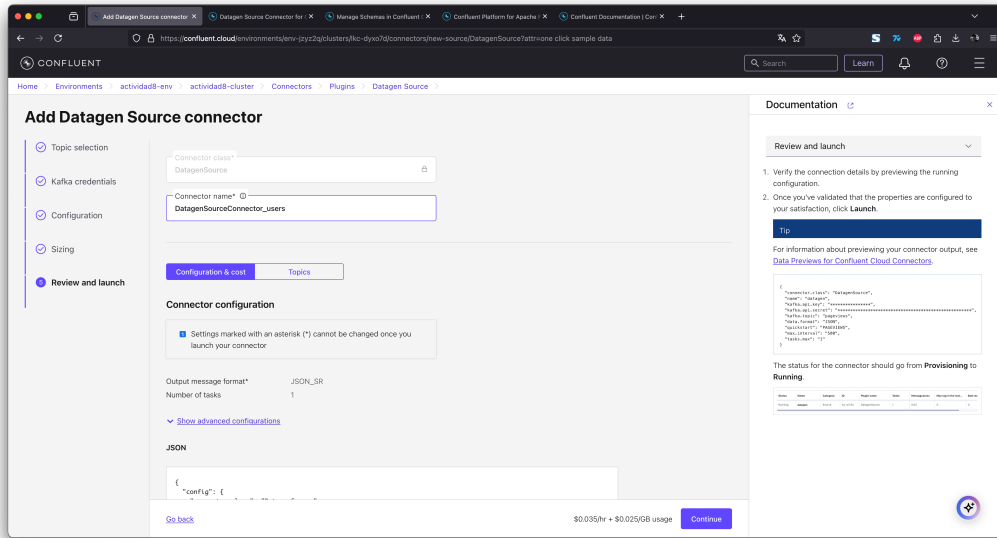
Generación de la API Key para autenticar al conector y permitir la recepción y envío de datos desde el clúster con aplicaciones externas



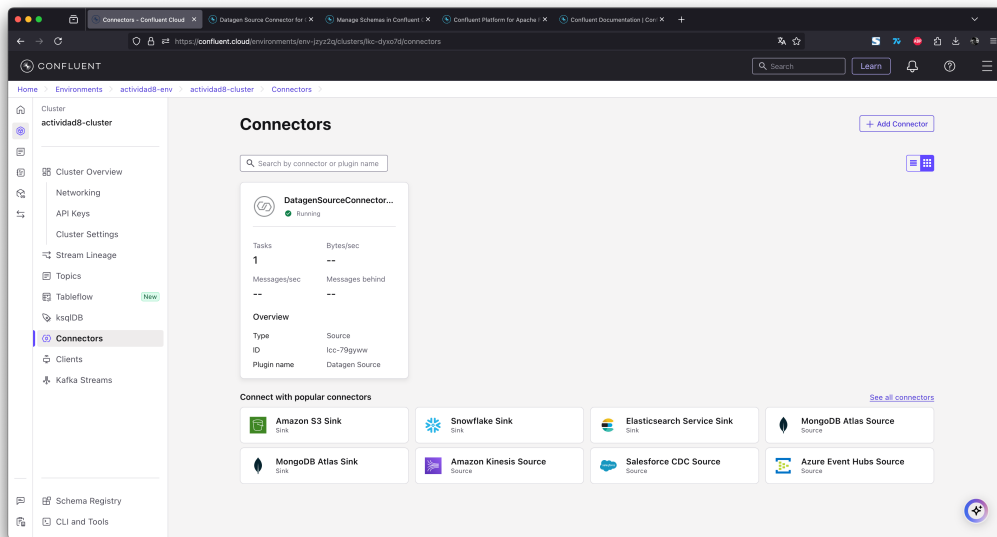
Selección del esquema de datos predefinido con formato de serialización JSON_SR para estructurar los mensajes producido



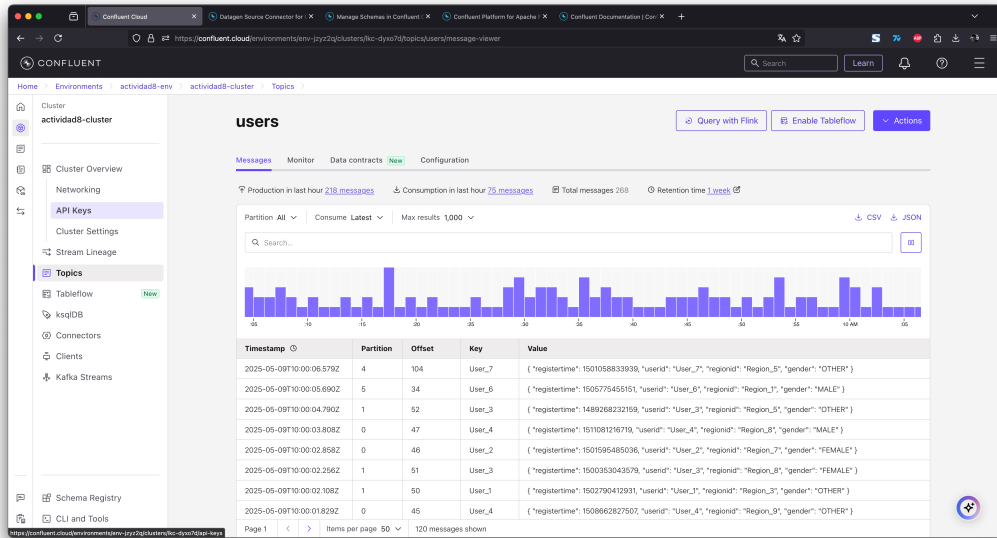
Configuración del *Sizing* del conector para definir la velocidad de generación de mensajes que se envían al topic



Revisión final antes de desplegar el conector



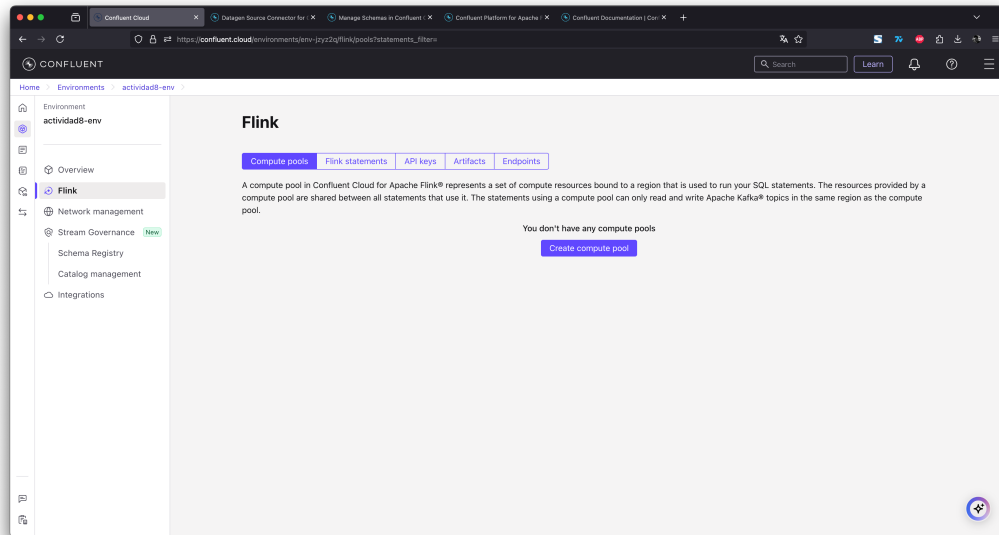
Estado activo del conector Datagen



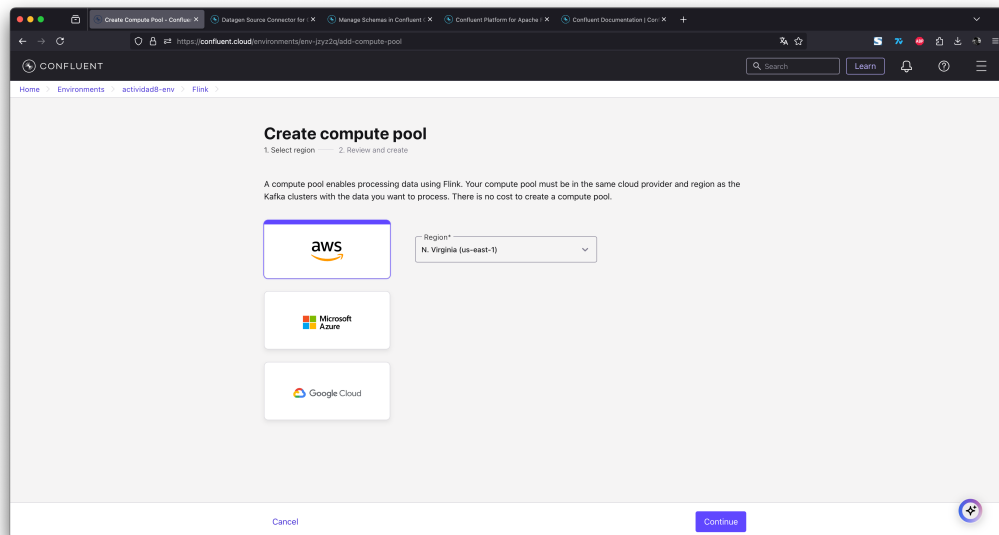
Comprobación en tiempo real de los datos generados en el topic 'users'

Consultas en tiempo real con Flink

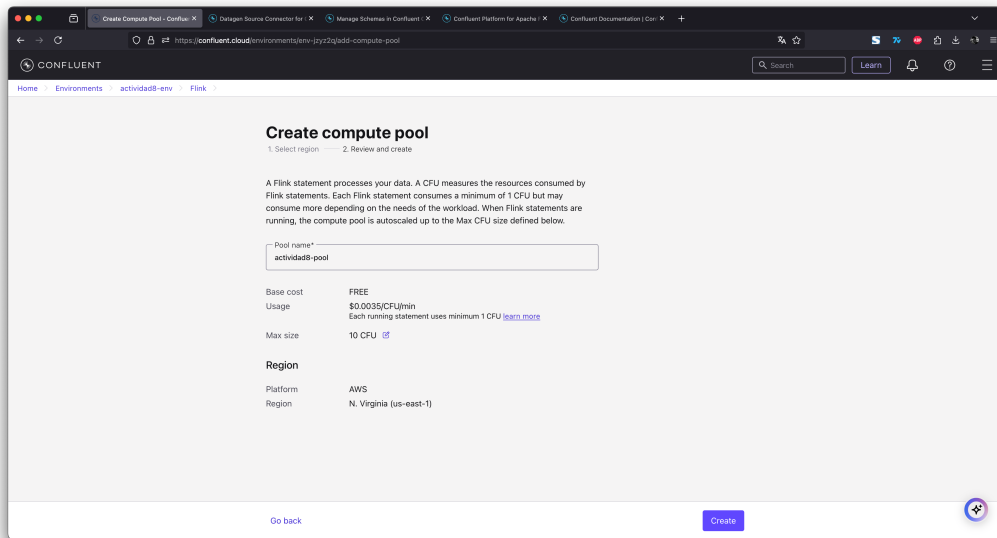
Finalmente, se configura un Compute Pool que permite ejecutar operaciones SQL en tiempo real mediante Flink. Una vez aprovisionado el recurso, se accede a Flink SQL Studio para consultar los datos del topic.



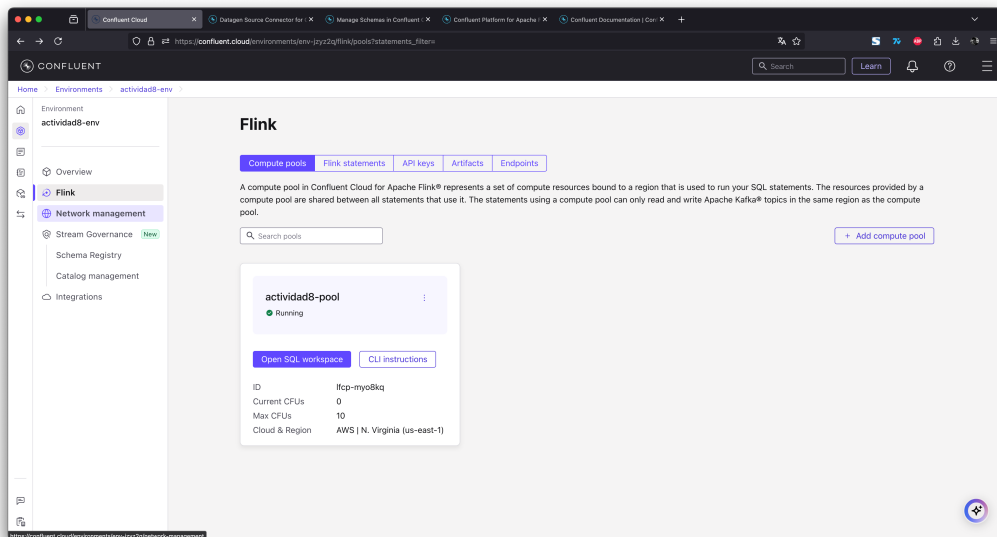
Página de inicio para la creación de un *Compute Pool*



Creación del Compute Pool utilizando como proveedor AWS y la región N.Virgina(us-east-1)



Resumen de configuración



Confirmación visual del *Compute Pool* desplegado y funcionando

workspace-2025-05-09-101537

ASIER ORTIZ AWS | us-east-1

User Account actividad8-pool Running

Catalog actividad8-env Database actividad8-cluster

```
1 SELECT * FROM users LIMIT 10;
```

START TIME: 2025-05-09T10:15:38Z02Z STATEMENT STATUS: Running STATEMENT NAME: workspace-2025-05-09- RESULTS: 10

key	registertime	userid	regionid	gender
x'557365725f38'	1497795951256	User_8	Region_4	MALE
x'557365725f38'	149747322968	User_8	Region_9	MALE
x'557365725f38'	1509440425827	User_8	Region_7	FEMALE
x'557365725f38'	1498868525327	User_8	Region_8	FEMALE
x'557365725f38'	1506295214972	User_8	Region_9	OTHER
x'557365725f38'	1492731090765	User_8	Region_4	MALE
x'557365725f38'	1492536588091	User_8	Region_7	FEMALE
x'557365725f38'	1499270962384	User_8	Region_3	FEMALE
x'557365725f38'	1513583926050	User_8	Region_3	MALE
x'557365725f38'	1496422571363	User_8	Region_4	OTHER

Last saved at 9/5/25, 12:17

Consulta para visualizar los 10 registros más recientes

workspace-2025-05-09-101537

ASIER ORTIZ AWS | us-east-1

User Account actividad8-pool Running

Catalog actividad8-env Database actividad8-cluster

```
1 SELECT gender, COUNT(*) AS total_users
2 FROM users
3 GROUP BY gender;
```

START TIME: 2025-05-09T10:16:29Z02Z STATEMENT STATUS: Running STATEMENT NAME: workspace-2025-05-09- RESULTS: 3

gender	total_users
OTHER	853
MALE	831
FEMALE	876

Last saved at 9/5/25, 12:19

Consulta agrupada por género, contando el total de usuarios por cada categoría

workspace-2025-05-09-101...

workspace-2025-05-09-101537

ASIER ORTIZ AWS | us-east-1

User Account actividad8-env Running

Catalog actividad8-env Database actividad8-cluster

```
1 SELECT regionid, COUNT(*) AS users_per_region
2 FROM users
3 GROUP BY regionid;
```

START TIME: 2025-05-09T10:20:58.668832Z STATEMENT STATUS: Running STATEMENT NAME: workspace-2025-05-09-...

RESULTS: 9 Stop

regionid	users_per_region
Region_2	291
Region_8	290
Region_4	314
Region_9	305
Region_7	326
Region_1	302
Region_5	303
Region_6	318
Region_3	310

users

- Schema (6)
 - key VARBINARY
 - registertime BIGINT
 - userid VARCHAR
 - regionid VARCHAR
 - gender VARCHAR
 - rowtime TIMESTAMPTZ
- Options (10)

Last saved at 9/5/25, 12:20 1/20 cells

Consulta agrupada por región, mostrando en tiempo real la cantidad de usuarios por ubicación

3. Parte 2 – Implementación con Python

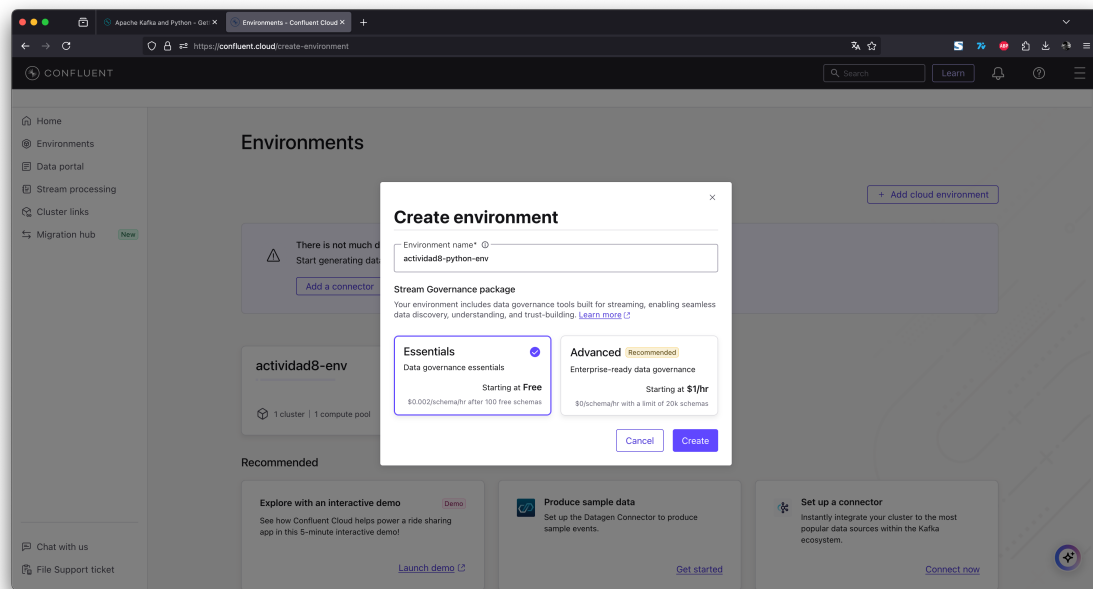
En esta segunda parte, sigo los pasos de la [guía oficial de Confluent](#) para crear el entorno desde la web y conectar mis scripts de Python a un clúster remoto en Confluent Cloud.

Creación del entorno, cluster, topic y API Keys

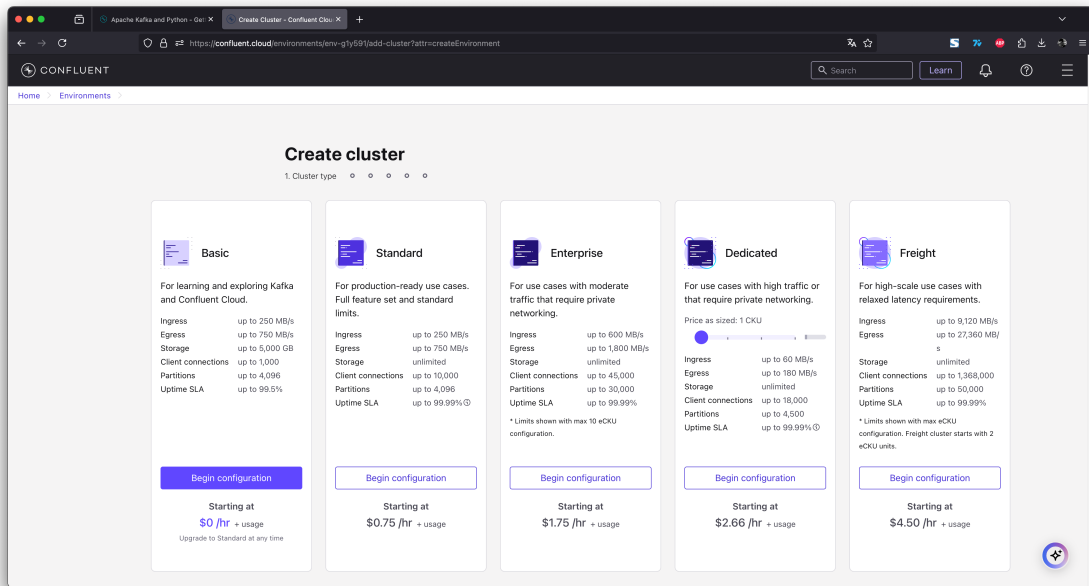
He creado un entorno denominado ‘actividad8-python-env’ y un clúster asociado llamado ‘actividad8-python-cluster’, donde he configurado un topic llamado ‘pec-topic1-asier’ con dos particiones. Aunque el enunciado pedía un factor de replicación de 1, la plataforma sólo me permitía seleccionar un valor fijo de 3, sin posibilidad de modificarlo.

Respondiendo a la pregunta del enunciado; no sería posible configurarlo con un factor de replicación de 2 en este entorno porque sólo hay un nodo disponible, por lo que no se podrían distribuir las réplicas entre distintos brokers.

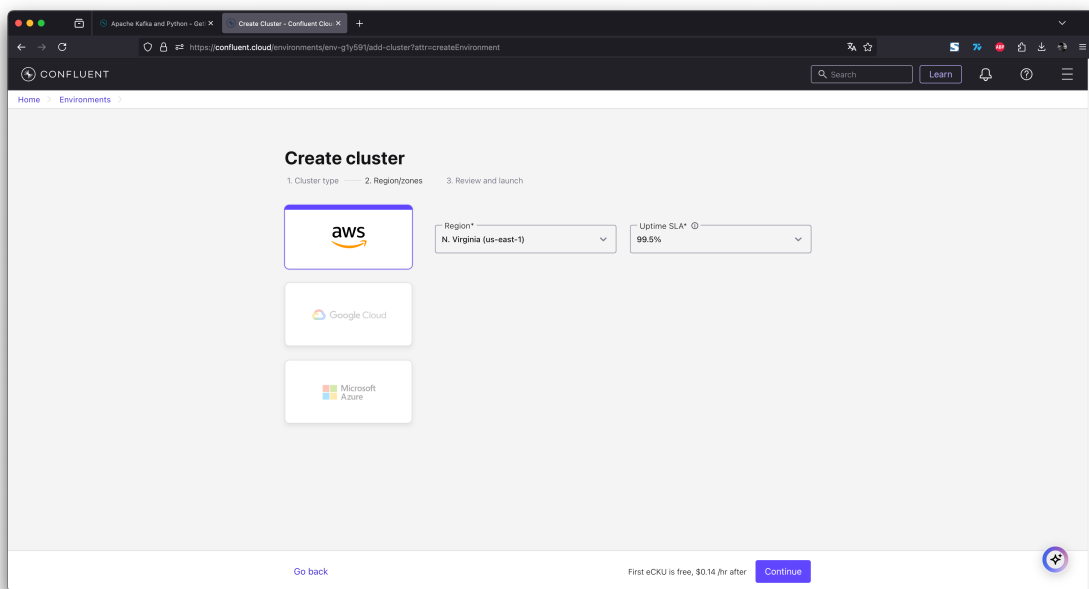
Para finalizar, he generado una API Key y he descargado las credenciales necesarias.



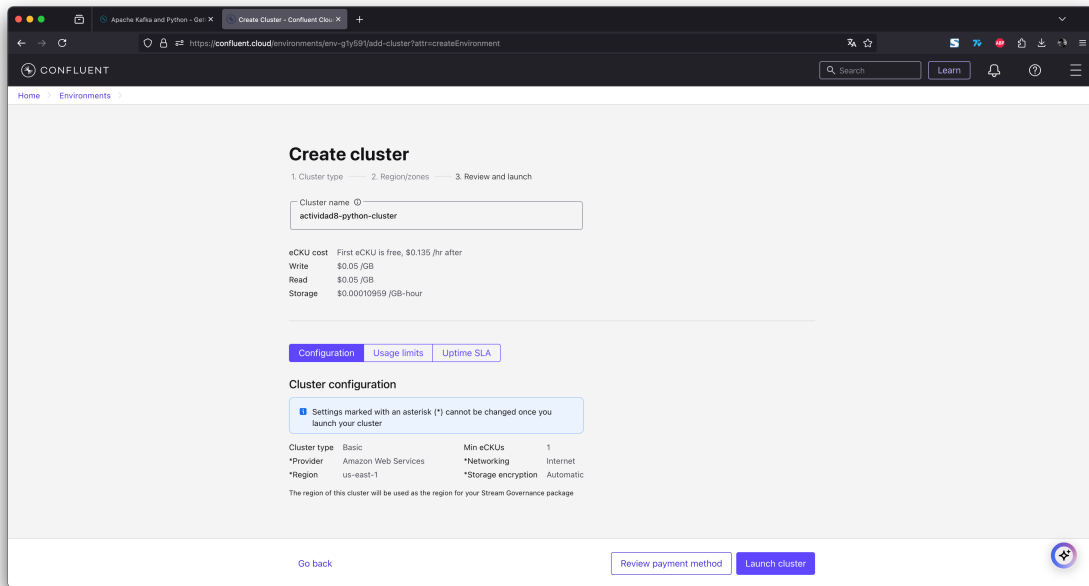
Creación de un nuevo entorno para el ejercicio



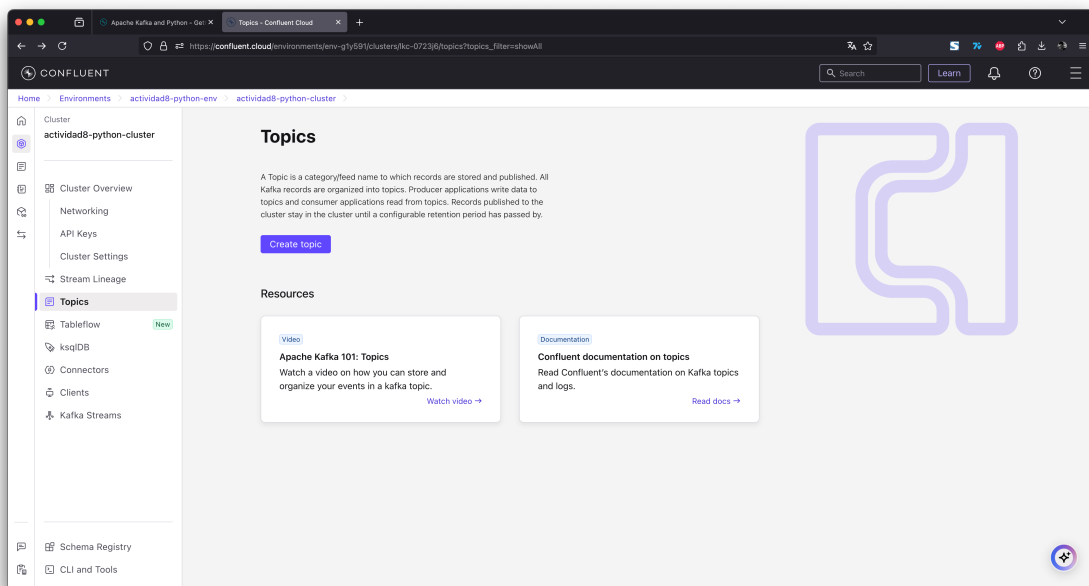
Página de inicio de creación del clúster



Selección del proveedor donde se alojará el cluster



Resumen de configuración final antes de lanzar el clúster



Acceso a la selección de topics dentro del cluster para la iniciar la creación de un nuevo topic

The screenshot shows the 'Create Topic' configuration page in the Confluent Cloud interface. The 'General' section has 'Topic name' set to 'pec-topic1-asier' and 'Partitions' set to '2'. The 'Storage' section has 'Cleanup policy' set to 'Delete', 'Retention time' set to '1 week', and 'Retention size' set to 'Infinite'. There is a checkbox for 'Enable infinite retention for this topic' which is currently unchecked. Below this, a text box explains that retaining data infinitely allows access to historical and current data. The 'Message size' section has 'Maximum message size in bytes' set to '2097164 bytes'. The 'Other Settings' section shows 'cluster' as 'actividad8-python-cluster', 'replication.factor' as '3', and 'min.insync.replicas' as '2'. At the bottom, there are 'Cancel' and 'Save & create' buttons.

General

Topic name* Partitions*

Storage

Cleanup policy

Retention time Retention size

☐ Enable infinite retention for this topic

Retain your data in Kafka infinitely to access historical and current data all in one place for data analysis, regulatory purposes, or other use cases. [Learn more](#)

Message size

Maximum message size in bytes

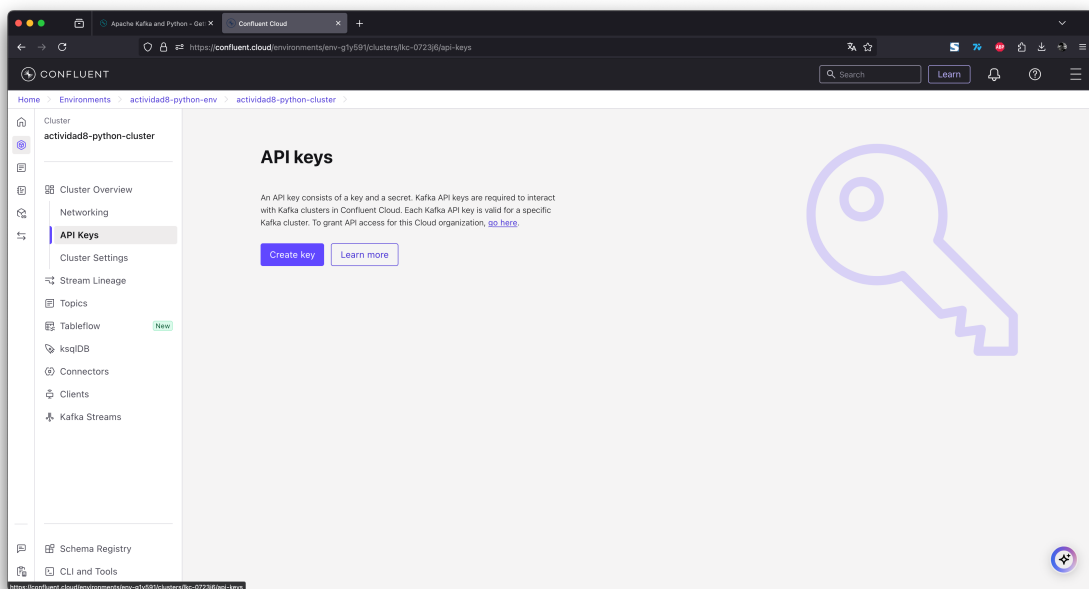
Other Settings

cluster
replication.factor
min.insync.replicas

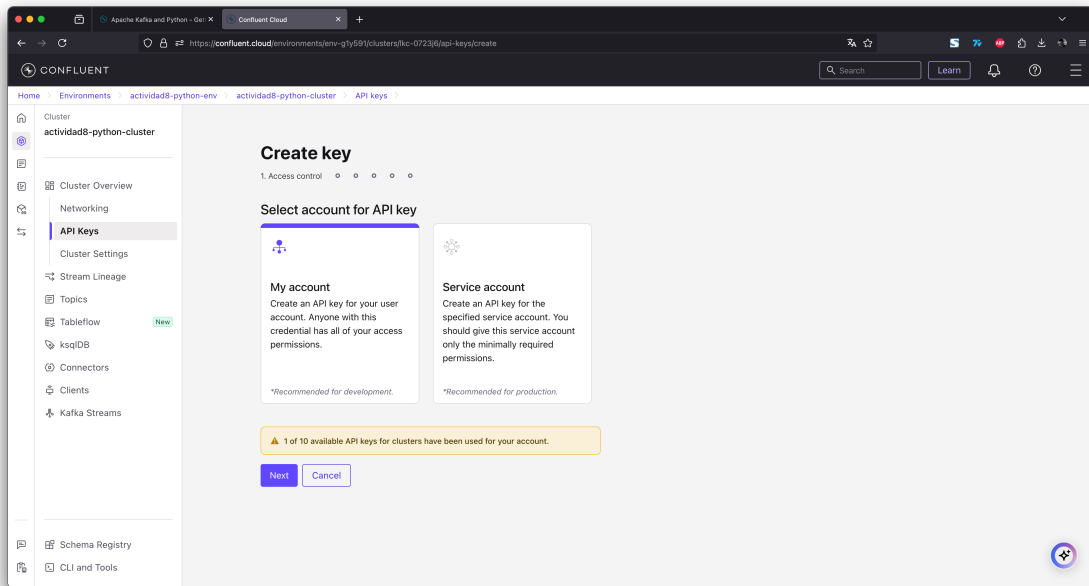
[Use topic defaults](#)

[Cancel](#) [Save & create](#)

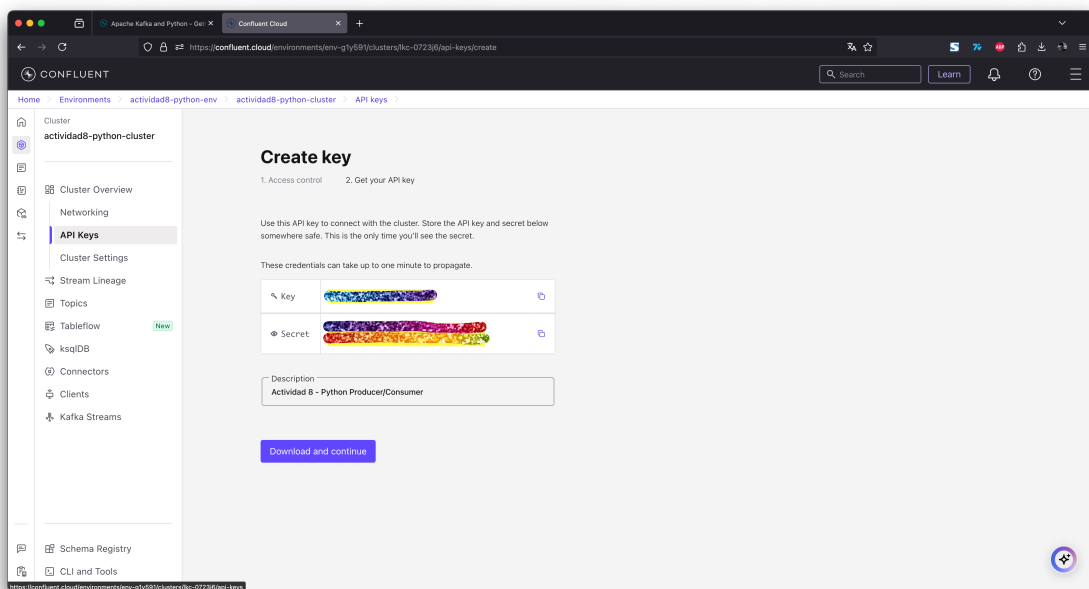
Configuración del topic pec-topic1-asier con 2 particiones



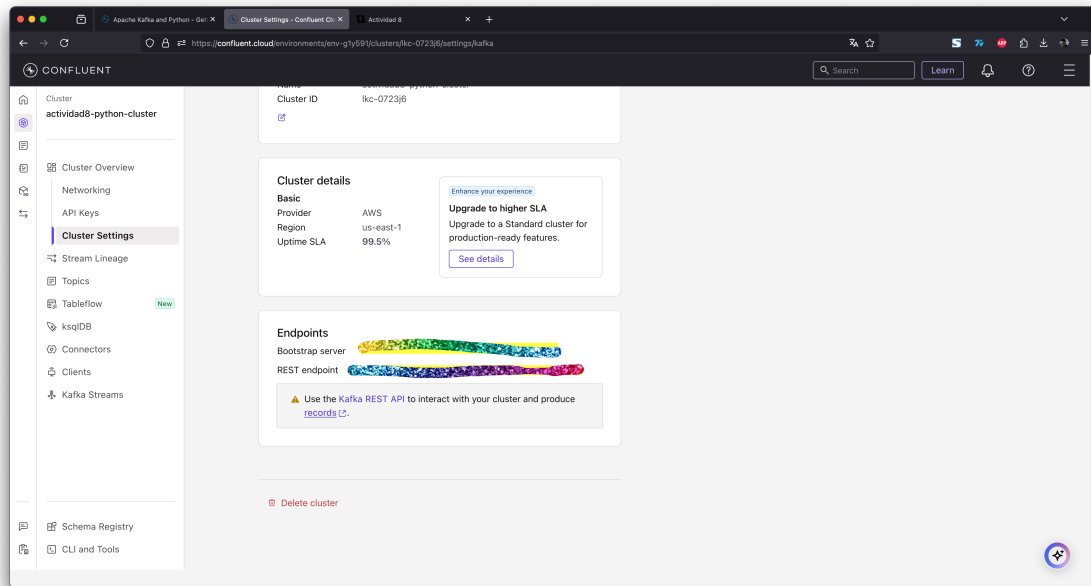
Acceso a la selección de API Keys



Selección del tipo de cuenta para la API Key



Generación y descarga de la Key y del Secret



Visualización de endpoints del cluster

Ejecución de scripts desde Python

Una vez configurado el entorno y generadas las claves de acceso, se ejecutan los scripts Python para producir y consumir mensajes.

- 'producer.py' genera 10 eventos con claves y valores aleatorios y los envía al topic 'pec-topic1-asier'.

```
producer.py
1  #!/usr/bin/env python
2
3  from random import choice
4  from confluent_kafka import Producer
5  from utils import load_config
6  import os
7
8  # Cargar configuración desde .env
9  conf = load_config()
10 topic = os.getenv("TOPIC", "pec-topic1-asier")
11
12 # Crear instancia del productor
13 producer = Producer(conf)
14
15 # Callback para confirmar entregas
16 def delivery_callback(err, msg):
17     if err:
18         print(f"[ERROR] Fallo al entregar mensaje: {err}")
19     else:
20         print(f"[OK] Enviado a {msg.topic():} key = {msg.key().decode('utf-8')}, value = {msg.value().decode('utf-8')}")
21
22 # Listas de ejemplo
23 user_ids = ['eabara', 'jsmith', 'sgarcia', 'jbernard', 'htanaka', 'awalther']
24 products = ['book', 'alarm clock', 't-shirts', 'gift card', 'batteries']
25
26 # Enviar 10 mensajes aleatorios
27 for _ in range(10):
28     user_id = choice(user_ids)
29     product = choice(products)
30     producer.produce(topic, key=user_id, value=product, callback=delivery_callback)
31     producer.poll(0) # Ejecuta los callbacks pendientes
32
33 producer.flush() # Espera hasta que todos los mensajes hayan sido entregados
34
35
36
UTF-8 LF 4 spaces Python 3.9 (ai-course-notebooks)
```

```
Terminal Local x Local (2) x Local (3) x + v
~/ai-course-notebooks/actividad8 on main python3 producer.py
%4|1746876640.614|CONFWARN|rdkafka#producer-1| [thrd:app]: Configuration property group.id is a consumer property and will be ignored by this producer instance
%4|1746876640.614|CONFWARN|rdkafka#producer-1| [thrd:app]: Configuration property auto.offset.reset is a consumer property and will be ignored by this producer instance
%6|1746876641.903|GETSUBSCRIPTIONS|rdkafka#producer-1| [thrd:main]: Telemetry client instance id changed from AAAAAAAAAAAAAAAAAAAAAA to 9Lkk++LXR30QJdTiyhDAMQ
[OK] Enviado a pec-topic1-asier: key = htanaka, value = alarm clock
[OK] Enviado a pec-topic1-asier: key = htanaka, value = book
[OK] Enviado a pec-topic1-asier: key = sgarcia, value = book
[OK] Enviado a pec-topic1-asier: key = htanaka, value = alarm clock
[OK] Enviado a pec-topic1-asier: key = jsmith, value = gift card
[OK] Enviado a pec-topic1-asier: key = eabara, value = book
[OK] Enviado a pec-topic1-asier: key = jbernard, value = gift card
[OK] Enviado a pec-topic1-asier: key = jbernard, value = gift card
[OK] Enviado a pec-topic1-asier: key = jbernard, value = gift card
[OK] Enviado a pec-topic1-asier: key = jbernard, value = gift card
[OK] Enviado a pec-topic1-asier: key = jbernard, value = gift card
~/ai-course-notebooks/actividad8 on main
```

- 'consumer.py' se conecta al mismo topic y lee todos los mensajes, mostrando su clave y valor.

```
consumer.py
consumer.py x
1  #!/usr/bin/env python
2
3  from confluent_kafka import Consumer, KafkaError
4  from utils import load_config
5  import os
6
7  # Cargar configuración y topic
8  conf = load_config()
9  topic = os.getenv("TOPIC", "pec-topic1-asier")
10 consumer_id = os.getenv("CONSUMER_ID", "X")
11
12 # Crear instancia del consumidor
13 consumer = Consumer(conf)
14 consumer.subscribe([topic])
15
16 print(f"[{consumer_id}] Escuchando mensajes en el topic '{topic}'...")
17
18 try:
19     while True:
20         msg = consumer.poll(1.0)
21         if msg is None:
22             print(f"[{consumer_id}] Esperando mensajes...")
23             continue
24         if msg.error():
25             if msg.error().code() != KafkaError._PARTITION_EOF:
26                 print(f"[{consumer_id}] Error: {msg.error()}")
27                 continue
28
29         key = msg.key().decode('utf-8') if msg.key() else None
30         value = msg.value().decode('utf-8') if msg.value() else None
31
32         print(f"[{consumer_id}] key = {key:10} | value = {value}")
33
34 except KeyboardInterrupt:
35     print(f"[{consumer_id}] Interrumpido por el usuario.")
36 finally:
37     consumer.close()
UTF-8 LF 4 spaces Python 3.9 (ai-course-notebooks)
```

```
Terminal Local x Local (2) x Local (3) x + v
~/Documents/ai-course-notebooks/actividad8 on main python3 consumer.py
[X] Escuchando mensajes en el topic 'pec-topic1-asier'...
[X] Esperando mensajes...
%6|1746876686.267|GETSUBSCRIPTIONS|rdkafka#consumer-1| [thrd:main]: Telemetry client instance id changed from AAAAAAAAAAAAAAAAAAAAAA to K
vm7MVxbRWq8mkQvg/2B0w
[X] Esperando mensajes...
[X] Esperando mensajes...
[X] key = awalther | value = batteries
[X] key = awalther | value = gift card
[X] key = htanaka | value = t-shirts
[X] key = jsmith | value = t-shirts
[X] key = sgarcia | value = t-shirts
[X] key = jbernard | value = gift card
[X] key = eabara | value = gift card
[X] key = eabara | value = t-shirts
[X] key = eabara | value = book
[X] key = eabara | value = alarm clock
[X] Esperando mensajes...
[X] Esperando mensajes...
[X] Esperando mensajes...
[X] Esperando mensajes...
^C[X] Interrumpido por el usuario.
~/Documents/ai-course-notebooks/actividad8 on main took 9s ai-course-notebooks base at 13:30:13
```

- 'group_consumer.py' lanza múltiples instancias de 'consumer.py' para simular un grupo de consumidores distribuidos.

```
group_consumer.py
group_consumer.py x
1 import multiprocessing
2 import subprocess
3 import os
4
5
6 def run_consumer(instance_id): 1 usage
7     print(f"Lanzando consumidor {instance_id}")
8     env = os.environ.copy()
9     env["CONSUMER_ID"] = str(instance_id)
10    env["PYTHONUNBUFFERED"] = "1" # Para mostrar salida en tiempo real
11    subprocess.run(args=["python3", "consumer.py"], env=env)
12
13
14 if __name__ == "__main__":
15     workers = []
16     for i in range(2): # Simula dos consumidores
17         p = multiprocessing.Process(target=run_consumer, args=(i,))
18         p.start()
19         workers.append(p)
20     for p in workers:
21         p.join()
22
```

UTF-8 LF 4 spaces Python 3.9 (ai-course-notebooks)

```
Terminal Local x Local (2) x Local (3) x + v
~/Documents/ai-course-notebooks/actividad8 on main python3 group_consumer.py
Lanzando consumidor 1
Lanzando consumidor 0
[1] Escuchando mensajes en el topic 'pec-topic1-asier'...[0] Escuchando mensajes en el topic 'pec-topic1-asier'...

[1] Esperando mensajes...
[0] Esperando mensajes...
%6|1746876697.117|GETSUBSCRIPTIONS|rdkafka#consumer-1 [thrd:main]: Telemetry client instance id changed from AAAAAAAAAAAAAAAAAAAAAA to /
7Wh1JSDQ1mZxcRU09sLGA
%6|1746876697.162|GETSUBSCRIPTIONS|rdkafka#consumer-1 [thrd:main]: Telemetry client instance id changed from AAAAAAAAAAAAAAAAAAAAAA to b
ZUy7rHAQP2mE3A0dF5YAq
[1] Esperando mensajes...
[0] Esperando mensajes...
[0] key = eabara | value = book
[0] key = jbernard | value = gift card
[0] key = jbernard | value = gift card
[0] key = jbernard | value = gift card
[0] key = jbernard | value = gift card
[1] key = htanaka | value = alarm clock
[1] key = htanaka | value = book
[1] key = sgarcia | value = book
[1] key = htanaka | value = alarm clock
[1] key = jsmith | value = gift card
[0] Esperando mensajes...
[1] Esperando mensajes...
[0] Esperando mensajes...
[1] Esperando mensajes...
[0] Esperando mensajes...
```

Esta implementación permite demostrar cómo Kafka reparte mensajes entre particiones, manteniendo la afinidad por clave. El código completo está disponible en el siguiente repositorio:

<https://github.com/asier-ortiz/ai-course-notebooks/tree/main/actividad8>